

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«НОВОСИБИРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ» (НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ, НГУ)

Факультет Механико-математический
Кафедра Программирования

Направление подготовки Математика и компьютерные науки

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА БАКАЛАВРА

Новиков Сергей Александрович

(Фамилия, Имя, Отчество автора)

Тема работы: Применение нейроэволюционных алгоритмов для
 моделирования когнитивных агентов

«К защите допущена»

Вр.и.о. зав. кафедрой,
к.ф.-м.н.

Емельянов П.Г. / _____
(фамилия, И., О.) (подпись, МП)

«....».....2026 г.

Научный руководитель

д.ф.-м.н.,
старший преподаватель
кафедры программирования

Пальянов А.Ю. / _____
(фамилия, И., О.) (подпись, МП)

«....».....2026 г.

Дата защиты: «....»2026 г.

Содержание

1. Введение	6
2. Постановка задачи	7
2.1. Контекст исследования	7
2.2. Цель исследования	7
2.3. Задачи исследования	8
2.4. Метрики исследования	8
2.5. Математическая формализация	10
2.6. Сравнимые подходы	11
2.7. Ожидаемые результаты	11
3. Предшествующие работы	12
3.1. Глубокое обучение с подкреплением на основе аппроксимации Q-функции (DQN)	12
3.1.1. Постановка задачи и мотивация	12
3.1.2. Основные идеи и архитектура метода	13
3.1.3. Результаты и выявленные ограничения	14
3.1.4. Связь с задачей моделирования когнитивных агентов	14
3.2. Эволюция нейронных сетей через расширение топологий .	15
3.2.1. Постановка задачи и мотивация	15
3.2.2. Основные идеи и архитектура NEAT	16
3.2.3. Связь с задачей моделирования когнитивного агента	18
4. Ход исследования	18

4.1.	Алгоритмический базис: Жизненный цикл NEAT	19
4.1.1.	Кодирование нейросети: генотип и фенотип	19
4.1.2.	Отказ от слоистой архитектуры в пользу произволь- ных графов	20
4.1.3.	Скращивание и решение проблемы «соперничающих соглашений»	21
4.1.4.	Классификация генов при выравнивании	22
4.1.5.	Вычисление функции приспособленности (Fitness Evaluation)	24
4.1.6.	Операторы эволюции: мутация и скрещивание . . .	25
4.1.7.	Видообразование (Speciation) на практике	26
4.2.	Программная реализация и особенности NEAT-модели . . .	27
4.2.1.	Параметры популяции и среды (Секция [NEAT]) . . .	27
4.2.2.	Инициализация генома и принцип минимализма (Секция [DefaultGenome])	28
4.2.3.	Управление мутациями и структурными из- менениями (Секция [DefaultSpeciesSet] и [DefaultStagnation])	29
4.2.4.	Видообразование и генетическая дистанция (Секция [DefaultReproduction])	29
4.2.5.	Селекция и элитизм (Секции [DefaultReproduction] и [DefaultStagnation])	30
4.3.	Условия проведения экспериментов	31
4.4.	Аппаратное обеспечение экспериментов	31
4.5.	Среда CartPole (Балансировка перевернутого маятника) . . .	33

4.5.1.	Пространство состояний	34
4.5.2.	Пространство действий	34
4.5.3.	Функция награды и условия завершения	35
4.5.4.	Конфигурация алгоритма NEAT для среды CartPole .	35
4.5.5.	Архитектура и гиперпараметры агента DQN для сре- ды CartPole	37
4.6.	Среда LunarLander (Посадка лунного модуля)	40
4.6.1.	Пространство состояний	40
4.6.2.	Пространство действий	41
4.6.3.	Функция награды и условия завершения	42
4.6.4.	Конфигурация алгоритма NEAT для LunarLander . .	43
4.6.5.	Архитектура и гиперпараметры агента DQN для сре- ды LunarLander	45
4.7.	Среда Ant (Ходьба муравья)	46
4.7.1.	Пространство состояний и действий	47
4.7.2.	Функция награды и условия завершения	48
4.7.3.	Конфигурация алгоритма NEAT для среды Ant-v4 .	48
5.	Результаты исследования	51
5.1.	Результаты в среде CartPole	53
5.1.1.	Эффективность использования выборки и скорость схождения	53
5.1.2.	Качество решения и архитектурная сложность	54
5.1.3.	Устойчивость к внешним возмущениям	54
5.1.4.	Практические выводы	55

5.2.	Результаты в среде LunarLander	55
5.2.1.	Сравнительный анализ архитектурной сложности и обучения	56
5.2.2.	Феномен «нейролоботомии» и проблема локальных оптимумов	56
5.2.3.	Устойчивость в условиях неопределенности и внешних шумов	57
5.2.4.	Специфика программной реализации	58
5.2.5.	Практические выводы	58
5.3.	Результаты в среде Ant	59
5.3.1.	Обоснование выбора модальности обучения	59
5.3.2.	Эволюция функции вознаграждения (Reward Shaping) и адаптация среды	60
5.3.3.	Практические выводы	61
6.	Направления дальнейших исследований	62
6.1.	Разработка гибридных мультимодальных моделей (CNN + NEAT)	62
6.2.	Интеграция алгоритмов в институтский симулятор популяций	63

1. Введение

В последние годы методы обучения с подкреплением (Reinforcement Learning, RL) стали одной из ключевых парадигм в области искусственного интеллекта. Их успех связан с возможностью решать сложные задачи взаимодействия агента со средой без явного задания правил поведения, а через формирование стратегий на основе опыта. Классические алгоритмы, такие как Q-Learning, Policy Gradient или методы на основе функции выигрыша (например, Cross-Entropy Method), доказали свою эффективность в ряде прикладных областей — от игр и робототехники до систем рекомендаций и оптимизации бизнес-процессов. Однако основным ограничением этих алгоритмов выступает высокая зависимость от параметризации модели и сложности настройки гиперпараметров, что влияет на качество и скорость обучения.

Одним из направлений, способных частично преодолеть этот барьер, выступают эволюционные алгоритмы, а именно семейства методов NEAT (NeuroEvolution of Augmenting Topologies). В отличие от традиционного градиентного обучения, NEAT позволяет не только находить параметры нейросетей, но и дает возможность архитектуре эволюционировать, формируя структуру, более подходящую для взаимодействия с конкретной средой. Это открывает возможности для гибридного подхода, в котором классические методы RL могут быть усилены нейроэволюционными техниками, обеспечивая баланс между оптимизацией параметров и автоматизированным поиском архитектур.

В данной работе будет рассмотрено исследование взаимосвязи меж-

ду традиционными методами RL и их комбинацией с NEAT в процессе обучения агентов.

2. Постановка задачи

2.1. Контекст исследования

Современные методы обучения с подкреплением (Reinforcement Learning, RL) демонстрируют высокую эффективность в решении задач взаимодействия агента со средой. Однако классические алгоритмы RL, такие как Q-Learning, Policy Gradient и Cross-Entropy Method, существенно зависят от параметризации модели и настройки гиперпараметров, что ограничивает их адаптивность. Альтернативный подход предлагают нейроэволюционные алгоритмы, в частности семейство методов NEAT (NeuroEvolution of Augmenting Topologies), которые позволяют не только оптимизировать веса нейросетей, но и плавно изменять их топологию / архитектуру, добавляя новые и удаляя некоторые уже имеющиеся нейроны, имитируя процесс эволюции.

2.2. Цель исследования

Провести сравнительное исследование эффективности классических методов обучения с подкреплением и нейроэволюционных алгоритмов при моделировании когнитивного агента в контролируемой среде.

2.3. Задачи исследования

1. Реализовать базовую модель обучения с подкреплением для решения задачи моделирования поведения когнитивного агента
2. Применить алгоритм NEAT к той же задаче для эволюции архитектуры нейросети агента
3. Провести комплексный сравнительный анализ моделей по метрикам, определенным ниже

2.4. Метрики исследования

Определение 2.1. *Скорость обучения определяет, насколько быстро алгоритм находит приемлемое решение*

Из-за различий в процедурах обучения (эпохи/поколения против эпизодов/шагов) будем использовать две шкалы:

1. **Количество взаимодействий со средой:** Общее число тактов симуляции (frame steps), потребовавшихся агенту для достижения порогового значения награды $R_{threshold}$ (например, 95% от максимально возможного или эталонного). Это самая честная метрика для сравнения RL и NEAT. Используем Формулу: $N_{steps} = \sum_{i=0}^E T_i$, где E – количество эпизодов/поколений до достижения успеха, T_i – длительность i -го эпизода/поколения.
2. **Время обучения:** Астрономическое время (в минутах), затраченное на обучение до достижения критерия успеха на фиксированном оборудовании. Учитывает накладные расходы алгоритмов (например,

обратное распространение для RL против мутаций/скрещивания для NEAT). Используем Формулу: $T_{train} = \sum_{i=0}^E t_i$, где t_i – время i -го эпизода/поколения.

Определение 2.2. *Качество решения* характеризует эффективность итоговой стратегии агента после завершения обучения.

1. **Средняя накопленная награда:** среднее значение суммарной награды за эпизод, вычисленное на валидационном наборе из $N = 100$ эпизодов с фиксированной стратегией. Используем формулу: $\bar{R} = \frac{1}{N} \sum_{i=1}^N R_i$
2. **Стабильность поведения:** Стандартное отклонение накопленной награды за эпизод на том же валидационном наборе. Используем формулу: $SD(R) = \sqrt{\frac{1}{N} \sum_{i=1}^N (R_i - \bar{R})^2}$

Определение 2.3. *Сложность модели* оценивает "интеллектуальную экономичность" полученного решения. Критически важна для NEAT, который стремится к минимализму.

1. **Количество параметров:** Общее число обучаемых параметров (весов) в итоговой нейросети агента. Используем формулу: $P = \sum_{l=1}^L (n_{l-1} + 1) \times n_l$, где L – количество слоев, n_l – число нейронов в l -м слое.
2. **Плотность связей:** Отношение числа активных связей к максимально возможному числу связей для данного количества нейронов. Показывает, насколько разреженной (эффективной) получилась архитектура. Используем формулу: $D = \frac{E_{active}}{E_{max}}$, где E_{active} – число активных связей, E_{max} – максимально возможное число связей.

Определение 2.4. Устойчивость – способность агента сохранять функциональность при изменении условий среды, на которых он не обучался явно.

1. **Устойчивость к шуму наблюдений:** Измеряется как процентное снижение средней накопленной награды $\bar{R}$ при добавлении гауссовского шума к входным данным агента. Используем формулу:

$$Robustness_{noise} = \frac{\bar{R}_{clean} - \bar{R}_{noisy}}{\bar{R}_{clean}} \times 100\%$$
2. **Адаптивность к изменениям среды:** Оценка способности агента сохранять производительность при изменении параметров среды (например, скорости движения объектов, количества препятствий). Измеряется как процентное изменение средней накопленной награды $\bar{R}$ при модификации среды. Используем формулу: $Robustness_{env} = \frac{\bar{R}_{original} - \bar{R}_{modified}}{\bar{R}_{original}} \times 100\%$

2.5. Математическая формализация

Среда моделируется как Марковский процесс принятия решений (МППР) $(\mathcal{S}, \mathcal{A}, \mathcal{P}, r, \gamma)$, где $\mathcal{S}$ – пространство состояний среды, $\mathcal{A}$ – пространство действий агента, $r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ – функция награды. Задача обучения с подкреплением формулируется как максимизация ожидаемого кумулятивного вознаграждения:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \gamma^t r(s_t, a_t) \right] \quad (2.1)$$

где π_θ – политика агента, параметризованная вектором θ , $\gamma \in [0, 1]$ – коэффициент дисконтирования, T – максимальная длина эпизода, $\tau =$

$(s_0, a_0, s_1, a_1, \dots, s_{T-1}, a_{T-1}, s_T)$ – траектория взаимодействия агента со средой. Подробнее МППР рассматривается в статьях [1] и [2]. В случае текущей постановки МППР мы как раз ссылаемся на [1]е

2.6. Сравнимые подходы

1. **Классический RL-метод:** Оптимизация параметров θ нейросети фиксированной архитектуры методом градиентного спуска/подъема на основе оценки $\nabla_{\theta} J(\theta)$
2. **Нейроэволюционный подход (NEAT):** Совместная оптимизация топологии сети $\mathcal{T}$ и весов θ эволюционным алгоритмом, минимизирующим сложность при максимизации $J(\theta, \mathcal{T})$

2.7. Ожидаемые результаты

Оценка преимуществ и недостатков каждого подхода в соответствии с предложенными метриками, рекомендации по выбору метода в зависимости от характеристик задачи моделирования когнитивных агентов.

3. Предшествующие работы

3.1. Глубокое обучение с подкреплением на основе аппроксимации Q-функции (DQN)

3.1.1. Постановка задачи и мотивация

В фундаментальной работе [2] коллектив авторов из DeepMind (V. Mnih et al.) исследует проблему создания универсального когнитивного агента, способного обучаться сложным стратегиям управления напрямую из высокоразмерных сенсорных данных (в данном случае — необработанных пикселей экрана).

До появления этой работы классические алгоритмы обучения с подкреплением (RL) успешно применялись лишь в средах с небольшим набором дискретных состояний. При попытке перенести их на сложные задачи (где состояние среды описывается тысячами пикселей) возникало «проклятие размерности». Для решения этой проблемы традиционно требовалось ручное конструирование признаков (feature engineering), что лишало агента универсальности. Попытки же объединить глубокие нейронные сети (глубокое обучение) и обучение с подкреплением сталкивались с серьезной проблемой неустойчивости и расходимости алгоритма из-за сильной корреляции последовательных состояний и постоянного изменения распределения данных в процессе обучения.

3.1.2. Основные идеи и архитектура метода

Для преодоления барьера между высокоразмерным визуальным входом и обучением стратегии авторы предложили алгоритм Deep Q-Network (DQN). В его основе лежит использование сверточной нейронной сети (CNN) для аппроксимации функции оптимальной ценности действий $Q^*(s, a)$, которая оценивает максимальную ожидаемую награду при выборе действия a в состоянии s .

Чтобы стабилизировать обучение нейросети методами градиентного спуска в условиях RL, авторы внедрили два критически важных механизма:

1. **Механизм воспроизведения опыта (Experience Replay):** В процессе взаимодействия со средой агент сохраняет свои переходы (состояние, действие, награда, следующее состояние) в специальный кольцевой буфер (Replay Memory). Обучение нейросети происходит не на последних полученных кадрах, а на случайных мини-батчах (mini-batches), извлеченных из этого буфера. Это разрушает временную корреляцию между обучающими примерами и сглаживает изменения в распределении данных, предотвращая катастрофическое забывание.
2. **Итеративное обновление целевой сети (Target Network):** Для вычисления ошибки (Loss) при обновлении весов используется отдельная, «замороженная» копия нейросети. Веса основной сети обновляются на каждом шаге, а веса целевой сети копируются из основной лишь раз в C шагов. Это устраняет эффект «погони за движущейся целью», когда обновления Q -значений приводят к неконтролируемым колебаниям градиентов.

3.1.3. Результаты и выявленные ограничения

Алгоритм DQN был протестирован на 49 играх платформы Atari 2600. Прорывным результатом стало то, что **одна и та же архитектура нейросети** с неизменными гиперпараметрами смогла достичь или превзойти уровень профессионального игрока-человека в большинстве игр. Агент продемонстрировал способность самостоятельно формировать сложные когнитивные репрезентации (например, находить оптимальные траектории или планировать действия на долгосрок).

Однако, несмотря на выдающиеся результаты, глубокое обучение с подкреплением на базе DQN имеет ряд фундаментальных ограничений:

- **Жестко заданная и избыточная архитектура:** Для успешной работы DQN потребовалась достаточно крупная сверточная сеть с миллионами параметров. Архитектура подбиралась авторами вручную. Из-за этого модель обладает низкой «интеллектуальной экономичностью».
- **Сложность оптимизации:** Процесс крайне чувствителен к шагу обучения (learning rate) и параметрам исследования среды.
- **Высокие требования к выборке (Sample Inefficiency):** Агенту требуются миллионы тактов симуляции (frame steps) для схождения к оптимальной политике, так как обучение «с нуля» через градиентный спуск идет крайне медленно на ранних этапах.

3.1.4. Связь с задачей моделирования когнитивных агентов

Статья [2] демонстрирует, как классический подход решает задачу моделирования когнитивного агента через градиентную аппроксимацию Q-

функции при фиксированной топологии мозга (нейросети) агента.

Анализ ограничений DQN формирует четкое обоснование исследовательских задач, поставленных в п. 2.3. В то время как классический RL тратит огромные вычислительные ресурсы на обновление весов в избыточной, вручную заданной нейросети, **алгоритмы семейства NEAT предлагают альтернативный путь**. NEAT решает проблему жесткой архитектуры, начиная с минимально возможной топологии и инкрементально усложняя ее только там, где это необходимо для выживания агента. Сравнение метрик (скорости схождения, количества параметров и устойчивости) между подходом, вдохновленным DQN, и алгоритмом NEAT является центральным элементом практической части данной работы.

3.2. Эволюция нейронных сетей через расширение топологий

3.2.1. Постановка задачи и мотивация

В своей работе [3] Stanley и Miikkulainen рассматривают нейроэволюцию (NE) как альтернативу классическим методам обучения с подкреплением, где вместо оценки функции ценности (value function) или градиентов политики напрямую реализуется эволюция поведения агента, заданного параметрами нейросети. Ключевой вопрос: можно ли получить преимущество, эволюционируя топологию сети вместе с весами, по сравнению с традиционными подходами, где оптимизируются только веса при фиксированной архитектуре.

Авторы указывают несколько проблем в классических подходах по

эволюции топологии и весов в нейронных сетях (TWEANN-подходы):

1. Проблема **соперничающих соглашений (competing conventions)** — одна и та же функция может быть закодирована разными перестановками скрытых нейронов, что делает кроссовер разрушающим. Кроссовером считаем операцию скрещивания информации из разных генов. Решение этой проблемы будет рассмотрено далее.
2. Сложность **сохранения инноваций**: новые структурные элементы (нейроны/связи) сначала ухудшают приспособленность, в результате чего агенты быстро вымирают без специальной защиты
3. Отсутствие механизма, который бы **поощрял минимальные архитектуры** и рост сложности “по мере необходимости”

3.2.2. Основные идеи и архитектура NEAT

NEAT предлагает три ключевых механизма решения этих проблем:

1. Исторические метки

Каждому новому гену (связи/нейрону) при структурной мутации присваивается уникальный номер инновации. Это позволяет при кроссовере однозначно “выравнивать” гены по происхождению и отличать:

- совпадающие внутри диапазона гены
- несоответствующие внутри диапазона
- и вне диапазона

За счёт этого кроссовер между разными топологиями становится осмысленным и не разрушает функциональные подструктуры.

2. Защита при кроссовере

Вводится метрика “генетической дистанции” между геномами на основе числа disjoint/excess-генов и различий в весах. Популяция разбивается на виды (species), и внутри вида применяется распределение приспособленности (fitness sharing). Это:

- ограничивает размер каждого вида
- не даёт одному “удачному” шаблону подавлять остальные
- создаёт “защищённые ниши” для особей с новыми структурами, которым требуется несколько поколений, чтобы стать полезными

3. Инкрементальный рост из минимальной архитектуры

Вместо старта с большого случайного множества топологий NEAT начинает с **минимальных сетей без скрытых нейронов** (только входы–выходы) и постепенно добавляет скрытые нейроны и связи только тогда, когда это даёт выигрыш по приспособленности.

Это фактически минимизирует размерность пространства поиска на всех этапах эволюции и делает обучение существенно более эффективным.

Также подробно описаны два типа структурных мутаций:

- добавление связи между двумя ранее не связанными узлами;
- расщепление существующей связи добавлением нового скрытого узла (при этом старая связь дезактивируется, а новые получают веса, минимизирующие резкое ухудшение функции).

3.2.3. Связь с задачей моделирования когнитивного агента

Результаты Stanley и Miikkulainen [3] обосновывают выбор NEAT как базового нейроэволюционного алгоритма для моделирования когнитивного агента. NEAT:

- естественно работает в задачах **обучения с подкреплением**, где поведение агента задаётся нейросетью
- хорошо справляется с задачами с частичной **наблюдаемостью и памятью** (за счёт рекуррентных связей и эволюции структуры)
- позволяет **избежать ручного подбора архитектуры**, что является серьёзной проблемой в классическом RL-подходе, где архитектура фиксируется заранее.

Это даёт чёткую исследовательскую гипотезу:

”Эволюция топологии нейросети когнитивного агента методом NEAT может обеспечить более высокую эффективность обучения (скорость, устойчивость, компактность модели) по сравнению с классическим RL-подходом при использовании фиксированной архитектуры сети.”

4. Ход исследования

Прежде чем переходить к описанию экспериментов в симулируемых средах, необходимо формализовать алгоритмический жизненный цикл нейроэволюционного подхода. В данном разделе подробно рассматривается внутренняя архитектура алгоритма NEAT и механизмы его работы, которые применялись при обучении когнитивных агентов.

4.1. Алгоритмический базис: Жизненный цикл NEAT

В отличие от классического обучения с подкреплением (RL), где оптимизируются только веса заранее заданной графовой структуры, NEAT управляет популяцией агентов, одновременно изменяя как веса, так и саму топологию графа. Процесс обучения представляет собой циклический эволюционный алгоритм, состоящий из нескольких ключевых этапов.

4.1.1. Кодирование нейросети: генотип и фенотип

Основополагающим принципом NEAT является разделение графа нейронной сети на генотип (информационный код, подверженный мутациям) и фенотип (рабочая нейронная сеть, управляющая когнитивным агентом в среде).

Генотип особи в NEAT состоит из двух списков генов:

- **Гены узлов (Node Genes):** определяют входы, выходы и скрытые нейроны агента. Каждый узел может иметь свою индивидуальную функцию активации (например, ReLU, Sigmoid или Tanh), что позволяет алгоритму подбирать оптимальную нелинейность для конкретной задачи.
- **Гены связей (Connection Genes):** описывают направленные синапсы между узлами. Каждый ген связи содержит информацию о начальном и конечном узле, текущем весе (weight), статусе активности (включена/выключена) и, что наиболее важно, номер инновации (innovation number).

Процесс превращения генотипа в фенотип (компиляция сети) происхо-

дит каждый раз, когда агента нужно протестировать в симуляции. Генотип подвергается мутациям и скрещиванию, а затем компилируется в конкретную архитектуру нейросети, которая управляет поведением агента в среде. Этот фенотип может быть как простой однослойной сетью, так и сложной рекуррентной архитектурой с несколькими скрытыми слоями и различными функциями активации.

4.1.2. Отказ от слоистой архитектуры в пользу произвольных графов

Важным концептуальным отличием алгоритма NEAT от традиционных методов глубокого обучения (Deep Learning) является полный отказ от парадигмы «слоев» (layers). В классических искусственных нейронных сетях, таких как многослойный перцептрон (MLP), архитектура строится строго послойно: сигнал синхронно передается от входного слоя к первому скрытому, затем ко второму и так далее, вплоть до выходного слоя. Вычисления при этом производятся структурными блоками через операции умножения матриц.

В отличие от этого, фенотип в алгоритме NEAT представляет собой **произвольный ориентированный граф**. Топология сети формируется абсолютно свободно в ходе эволюционных мутаций. Алгоритм не оперирует слоями, он работает исключительно с отдельными узлами и направленными связями (ребрами графа) между ними. Это приводит к ряду уникальных свойств моделируемого когнитивного агента:

- **Асимметричность маршрутизации сигналов:** Связь в графе NEAT может соединить входной сенсор напрямую с выходным моторным нейроном, в то время как сигнал от соседнего сенсора может обра-

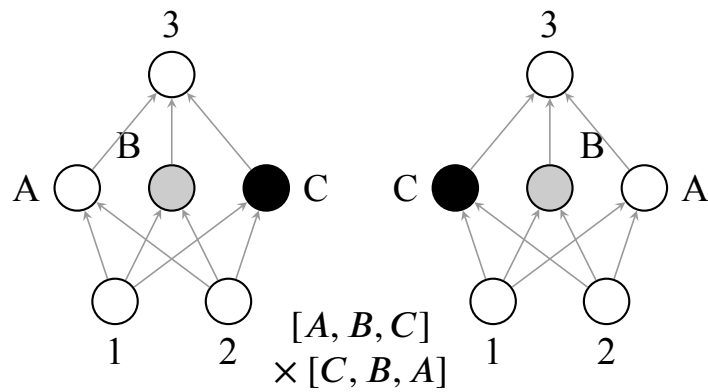
батываться через сложную цепочку из нескольких промежуточных узлов.

- **Отсутствие жесткой размерности:** Понятие «глубины сети» становится условным и определяется не количеством заданных слоев, а длиной максимального пути в графе от входного узла к выходному.
- **Точечное масштабирование:** Когда алгоритм принимает решение об усложнении архитектуры (структурная мутация), он не создает целый новый скрытый слой с множеством избыточных нейронов, как это делается в классическом подходе. Он лишь расщепляет одну конкретную связь, добавляя единичный вычислительный узел именно в ту часть графа, где требуется дополнительная нелинейность.

4.1.3. Скрещивание и решение проблемы «соперничающих соглашений»

Скрещивание (кроссовер) является одним из наиболее сложных операторов в нейроэволюции. Основная трудность классических подходов (TWEANN) заключалась в так называемой проблеме **соперничающих соглашений** (competing conventions), проиллюстрированная на рисунке 4.1. Она возникает, когда две нейронные сети кодируют одну и ту же функцию разными способами (например, разным расположением скрытых нейронов). При обычном случайном скрещивании таких сетей информация в геноме «перемешивается» деструктивно, что приводит к потере функциональных блоков и резкому падению приспособленности потомков.

Алгоритм NEAT решает эту проблему с помощью **исторических меток**



Потомки: $[A, B, A]$ $[C, B, C]$
 (разрушение структуры: потеря информации)

Рис. 4.1: Проблема конкурирующих соглашений

(innovation numbers). Каждому новому гену (связи или узлу), появившемуся в результате структурной мутации, присваивается уникальный идентификатор, который сохраняется за этой инновацией во всей популяции. Это позволяет алгоритму «выравнивать» геномы двух родителей, точно определяя, какие части их «цифрового мозга» имеют общее происхождение, а какие являются уникальными разработками конкретной эволюционной ветки.

4.1.4. Классификация генов при выравнивании

При создании потомка от двух родителей алгоритм сравнивает их списки генов связей. Гены распределяются по трем категориям на основе их номеров инноваций:

1. **Совпадающие гены (Matching Genes):** Гены с одинаковыми номерами инноваций у обоих родителей. Это означает, что данные связи были унаследованы от общего предка. Потомок получает этот ген от одного из родителей случайным образом, а веса связей усредняются

или также выбираются случайно.

2. **Разъединенные гены (Disjoint Genes):** Гены, которые присутствуют только у одного из родителей, но их номер инновации находится внутри диапазона номеров другого родителя. Это инновации, которые произошли в прошлом одной ветви, но отсутствуют в другой из-за того, что та ветвь пошла по иному пути развития.
3. **Избыточные гены (Excess Genes):** Гены, которые присутствуют только у одного из родителей и имеют номера инноваций, **превышающие максимальный номер** другого родителя. Это самые свежие структурные изменения, появившиеся совсем недавно в процессе эволюции одной из ветвей.

Пример для наглядности:

Представим Родителя А с генами [1,2,3,8,10] и Родителя Б с генами [1,2,5,6].

- Гены 1 и 2 — совпадающие (есть у обоих).
- Гены 3, 5, 6 — разъединенные (disjoint), так как они меньше максимального номера другого родителя (8 или 6 соответственно), но не имеют пары.
- Гены 8 и 10 — избыточные (excess), так как они больше максимального номера Родителя Б (6).

4.1.5. Вычисление функции приспособленности (Fitness Evaluation)

В контексте моделирования когнитивных агентов функция приспособленности (fitness) напрямую эквивалентна кумулятивной награде $J(\theta)$ из математической формализации RL (формула 2.1).

На каждом поколении (generation) для каждой особи в популяции выполняется следующий алгоритм:

1. Из генотипа компилируется фенотип (нейронная сеть прямого распространения или рекуррентная сеть).
2. Агент, управляемый этой сетью, помещается в среду (например, CartPole или LunarLander из фреймворка [4]).
3. Агент взаимодействует со средой до наступления терминального состояния (успех, провал или достижение лимита шагов).
4. Суммарная накопленная награда, полученная от среды, присваивается особи в качестве её показателя приспособленности (Fitness Score).

Именно этот показатель определяет вероятность особи передать свои гены следующему поколению.

Для того чтобы предотвратить накопление «мусорных» структур от неудачных особей, в NEAT применяется строгое правило: разъединенные и избыточные гены наследуются только от более приспособленного родителя (имеющего более высокий Fitness Score). Если же родители имеют равную приспособленность, то уникальные гены передаются потомку от обоих родителей с некоторой вероятностью.

4.1.6. Операторы эволюции: мутация и скрещивание

После оценки всей популяции алгоритм формирует новое поколение с помощью двух механизмов:

- **Мутация:** Существует два типа мутаций:
 - Параметрические мутации случайным образом сдвигают веса существующих связей (аналог шага градиентного спуска в RL, но со случайным вектором) или меняют функции активации узлов.
 - Структурные мутации усложняют топологию. Это может быть добавление новой связи между двумя ранее не связанными узлами или добавление нового скрытого узла. При добавлении узла старая связь отключается, а вместо нее создаются две новые. При этом вес одной связи устанавливается равным 1, а второй — равным весу старой связи. Это минимизирует первоначальное падение эффективности сети (сохраняет функцию фенотипа неизменной в момент мутации).
- **Скрещивание (Crossover):** При создании потомка от двух родителей алгоритм выравнивает их гены связей по номерам инноваций. Совпадающие гены (matching genes) наследуются от обоих родителей случайным образом. Гены, присутствующие только у одного родителя (disjoint и excess гены), наследуются от родителя с более высоким показателем приспособленности.

4.1.7. Видообразование (Speciation) на практике

Новые структурные элементы часто кратковременно снижают приспособленность особи, из-за чего в стандартных эволюционных алгоритмах инновации быстро вымирают. Для защиты структурных инноваций NEAT разбивает популяцию на виды (species).

Разделение происходит на основе метрики генетической дистанции δ , которая вычисляется по формуле:

$$\delta = \frac{c_1 E}{N} + \frac{c_2 D}{N} + c_3 \cdot \overline{W} \quad (4.1)$$

где E — количество excess-генов, D — количество disjoint-генов, $\overline{W}$ — среднее различие в весах совпадающих генов, N — нормализующий фактор (обычно принимается равным размеру большей из двух родительских сетей), а c_1, c_2, c_3 — коэффициенты, регулирующие важность каждого компонента.

Если дистанция δ между особью и представителем вида меньше заданного порога $\delta_{threshold}$, то особь зачисляется в этот вид. Далее применяется механизм разделения приспособленности (explicit fitness sharing): скорректированная приспособленность особи рассчитывается путем деления её реальной приспособленности на размер её вида.

Благодаря этому когнитивные агенты конкурируют в основном внутри своих ниш (видов). Это не позволяет одной простой, но локально успешной топологии мгновенно захватить всю популяцию, давая сложным архитектурам время на оптимизацию своих новых весов. Инкрементальный рост сложности от минимальных сетей к более сложным («Complexifying») яв-

ляется ключом к эффективности NEAT при поиске оптимального мозга агента.

4.2. Программная реализация и особенности

NEAT-модели

Для проведения вычислительных экспериментов и моделирования когнитивных агентов в данной работе используется язык программирования Python и специализированная библиотека [5] (далее, *neat-python*). Выбор данного инструмента обусловлен его гибкостью, открытым исходным кодом и строгим соответствием оригинальной архитектуре алгоритма NEAT, предложенной авторами работы [3].

Поведение алгоритма, параметры популяции и вероятности мутаций задаются централизованно через специальный конфигурационный файл. Ниже представлен анализ ключевых секций конфигурации, использовавшейся в ходе экспериментов, и их связь с теоретическим базисом нейроэволюции.

4.2.1. Параметры популяции и среды (Секция [NEAT])

Эта секция определяет глобальные условия эволюционного процесса:

- **pop_size:** Размер популяции определен и зафиксирован в течение всего эксперимента. Это обеспечивает достаточное генетическое разнообразие для поиска оптимальных топологий, не перегружая при этом вычислительные мощности.
- **fitness_threshold:** Порог приспособленности. Эволюция (обучение) будет автоматически остановлена, если лучший агент в популяции до-

стигнет суммарной награды в установленное количество баллов, что считается критерием успешного решения поставленной когнитивной задачи.

4.2.2. Инициализация генома и принцип минимализма (Секция [DefaultGenome])

Данный блок отвечает за стартовую архитектуру «мозга» агента и напрямую реализует концепцию инкрементального роста («Complexifying»), описанную ранее:

- **Интерфейс агента:** Задано `num_inputs` и `num_outputs`, что соответствует количеству сенсорных входов и моторных выходов агента.
- **Минимальная топология:** Параметр `num_hidden` строго запрещает наличие скрытых слоев в нулевом поколении. Параметр `initial_connection = full_direct` указывает, что изначально сенсоры соединены с выходами напрямую. Алгоритм будет вынужден сам выращивать промежуточные нейроны в процессе эволюции.
- **Активация:** Выбранная функция активации (например, `sigmoid`) определяет нелинейность в работе нейронов. Это важно для способности агента моделировать сложные зависимости между входами и выходами.

4.2.3. Управление мутациями и структурными изменениями (Секция [DefaultSpeciesSet] и [DefaultStagnation])

Вероятности генетических изменений тонко настроены для баланса между исследованием новых решений (exploration) и эксплуатацией удачных находок (exploitation):

- **Структурные мутации:** Реализованы вероятности добавления новой связи (add_connection_prob), добавления нового нейрона (add_node_prob), и удаления существующих связей и нейронов (delete_connection_prob, delete_node_prob). Эти параметры определяют темп роста сложности топологии и позволяют алгоритму адаптироваться к сложности задачи.
- **Параметрические мутации:** Вероятности мутации весов (weight_mutation_prob), а также их диапазон изменений (weight_replace_prob, weight_perturb_probs) обеспечивают гибкость в оптимизации параметров сети, не нарушая при этом её структурную целостность.

4.2.4. Видообразование и генетическая дистанция (Секция [DefaultReproduction])

Для защиты структурных инноваций в соответствии с формулой 4.1 настроены коэффициенты расчета генетической дистанции δ :

- **compatibility_excess_coefficient:** Устанавливает высокий штраф c_1 за избыточные (excess) гены, что делает акцент на различиях в топологии между сетями.

- **compatibility_disjoint_coefficient:** Задаёт высокий штраф c_2 за наличие разъединённых (disjoint) и избыточных генов, делая акцент на топологических различиях сетей.
- **compatibility_weight_coefficient:** Устанавливает низкий штраф c_3 за различия в весах, что позволяет некоторую вариативность в параметрах внутри одного вида.
- **compatibility_threshold:** Порог δ_t для определения принадлежности к одному виду. Этот параметр критически важен для поддержания разнообразия в популяции и защиты инноваций от преждевременного вымирания.

4.2.5. Селекция и элитизм (Секции [DefaultReproduction] и [DefaultStagnation])

Процесс формирования нового поколения регулируется строгими правилами отбора:

- **survival_threshold:** Процент лучших особей, которые будут допущены к размножению. Это обеспечивает отбор только наиболее приспособленных агентов, стимулируя эволюцию эффективных топологий.
- **elitism:** Количество лучших особей, которые сохраняются без изменений в следующее поколение. Это гарантирует сохранение лучших найденных решений и предотвращает их потерю из-за случайных мутаций.
- **Защита от стагнации:** Параметры `species_elitism` и `max_stagnation` обеспечивают механизмы для предотвращения застоя в эволюции,

поддерживая динамичное развитие популяции.

4.3. Условия проведения экспериментов

Для проведения вычислительных экспериментов и оценки эффективности нейроэволюционных алгоритмов использовался фреймворк `gymnasium` — современный стандарт в области обучения с подкреплением (RL), предоставляющий унифицированный программный интерфейс для симуляции различных физических и логических задач.

Каждая среда в данном фреймворке строго формализована в терминах марковского процесса принятия решений (МППР), что позволяет объективно сравнивать агентов, обученных методами классического RL и алгоритмом NEAT. Первым этапом тестирования стала базовая задача управления в непрерывном пространстве состояний. Все работы проводятся в стандартном Github репозитории¹.

4.4. Аппаратное обеспечение экспериментов

Для обеспечения прозрачности исследования и возможности воспроизведения полученных результатов необходимо зафиксировать конфигурацию вычислительного стенда, на котором производилось обучение и тестирование когнитивных агентов.

Поскольку нейроэволюционный алгоритм NEAT начинает поиск с минимальных топологий и наращивает их инкрементально, он не требует применения специализированных графических ускорителей (GPU) или

¹<https://github.com/Isn0v/neat-training>

тензорных процессоров (TPU), в отличие от классических избыточных архитектур глубокого обучения (например, архитектуры DQN с миллионами параметров). Основная вычислительная нагрузка при работе алгоритма ложится на центральный процессор из-за необходимости параллельной компиляции сотен индивидуальных графов (фенотипов) и симуляции физической среды для каждой особи в популяции.

В связи с этим вычислительные эксперименты проводились на локальной рабочей станции со следующими аппаратными и программными характеристиками:

- **Центральный процессор (CPU):** Intel Core i5-12500H (базовая тактовая частота 2.5 ГГц, поддержка многопоточных вычислений на 12 ядрах для параллельной оценки приспособленности в независимых эпизодах).
- **Оперативная память (RAM):** 16 ГБ, что полностью удовлетворяет требованиям к хранению истории мутаций, буферов воспроизведения опыта (в случае RL-baseline) и объектов среды *gymnasium*.
- **Операционная система:** Семейство Linux (Ubuntu 24.04 LTS), обеспечивающее стабильную работу с системными прерываниями и минимальные накладные расходы при аллокации памяти для процессов симуляции.
- **Программная среда:** Язык программирования Python 3.12.3, фреймворк для симуляции физических сред *gymnasium*, а также открытая библиотека *neat-python* для реализации эволюционного цикла.

4.5. Среда CartPole (Балансировка перевернутого маятника)

CartPole — это классическая задача теории управления, описанная в [6]. Она представляет собой тележку (каретку), которая может двигаться по одномерному треку без трения. К каретке с помощью шарнира прикреплен неустойчивый шест (маятник), центр тяжести которого находится выше точки опоры. Задача когнитивного агента заключается в том, чтобы не дать шесту упасть, прикладывая импульсы силы к каретке.

С точки зрения когнитивного моделирования, сложность задачи состоит в том, что агент изначально не знает законов физики (гравитации, инерции, массы объектов). Он должен самостоятельно сформировать внутреннее представление динамики системы исключительно на основе поступающих числовых сигналов.

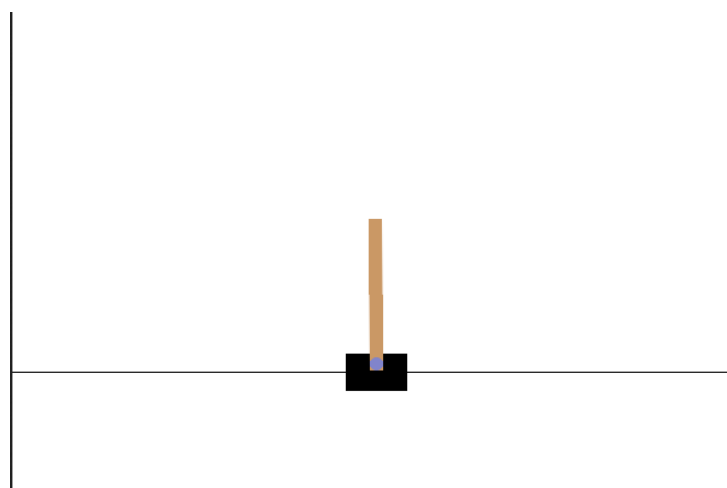


Рис. 4.2: Вид среды CartPole.

4.5.1. Пространство состояний

Сенсорный аппарат агента крайне ограничен. В каждый момент времени t среда передает агенту вектор S_t , состоящий ровно из 4 непрерывных кинематических значений:

- **Позиция каретки:** Текущее положение тележки на треке (от -4.8 до 4.8 условных единиц).
- **Скорость каретки:** Линейная скорость перемещения тележки.
- **Угол отклонения шеста:** Текущий угол наклона маятника относительно идеальной вертикали (в радианах).
- **Угловая скорость шеста:** Скорость падения или выравнивания маятника.

Именно этот вектор подается на входные узлы нейронной сети (фенотипа) в процессе оценки приспособленности.

4.5.2. Пространство действий

Пространство возможных решений агента дискретно. На каждом шаге (фрейме) агент обязан выдать одну из двух команд:

- **0:** приложить фиксированный импульс силы, толкнув каретку влево.
- **1:** приложить фиксированный импульс силы, толкнув каретку вправо.

Важной особенностью является отсутствие действия «ничего не делать». Агент вынужден постоянно совершать микрокорректировки, что требует от нейросети быстрой реакции и точного балансирования между противоположными командами.

4.5.3. Функция награды и условия завершения

Сигнал подкрепления (она же приспособленность для NEAT) в этой среде максимально прост: агент получает награду +1 за каждый такт (шаг) времени, в течение которого шест остается в допустимом вертикальном положении.

Эпизод симуляции считается завершенным (наступает терминальное состояние) при выполнении любого из следующих условий:

1. **Падение:** Угол отклонения шеста θ превысил допустимый порог (как правило, $\pm 12^\circ$ или ≈ 0.2095 радиан от вертикали).
2. **Выход за границы:** Каретка уехала слишком далеко от центра экрана (позиция $x < -2.4$ или $x > 2.4$).
3. **Успех:** Достигнут лимит времени симуляции (в версии среды CartPole-v1 этот лимит составляет 500 шагов).

Таким образом, для признания агента полностью успешным в данной среде, его суммарная накопленная награда (Fitness Score) за один эпизод должна достичь значения 500. Это означает, что нейросеть агента сформировала устойчивую стратегию балансировки, способную неограниченно долго (в рамках лимита среды) поддерживать динамическое равновесие.

4.5.4. Конфигурация алгоритма NEAT для среды CartPole

Ниже приведены ключевые параметры, использованные в экспериментах:

4.5.4.1. Начальная топология и архитектура В соответствии с фундаментальным принципом алгоритма NEAT — комплексным усложнением (complexifying) — эволюционный процесс инициировался с простейшей архитектуры. Сеть не содержала скрытых слоев (`num_hidden = 0`). Входной слой состоял из 4 нейронов, принимающих вектор состояния среды: позицию каретки, ее линейную скорость, угол отклонения шеста от вертикали и его угловую скорость. Выходной слой включал 2 нейрона, аппроксимирующих уверенность агента в необходимости приложения силы влево или вправо.

На старте эволюции применялась полная связность прямого распространения (`initial_connection = full_direct`), что заставляло алгоритм на ранних этапах опираться исключительно на прямые зависимости между сенсорными данными и моторными реакциями. Выбор итогового дискретного действия агента осуществлялся путем вычисления максимума аргумента (`argmax`) по вектору выходных значений сети. В качестве базовой функции активации для узлов применялась функция ReLU (`activation_default = relu`).

4.5.4.2. Параметрические и структурные мутации: Для обеспечения эффективного баланса между исследованием пространства решений (exploration) и эксплуатацией найденных оптимумов (exploitation), конфигурационный файл² был настроен с акцентом на активную параметрическую адаптацию. Вероятность мутации весов связей была установлена на высоком уровне — 80% (`weight_mutate_rate = 0.8`), при этом сила мутационных сдвигов контролировалась параметром `weight_mutate_power = 0.5`.

²<https://github.com/Isn0v/neat-training/blob/master/environments/cartpole/neat/neat.cfg>

Структурные мутации, отвечающие за рост сети, были настроены таким образом, чтобы отдавать приоритет образованию новых связей над добавлением новых узлов. Так, вероятность добавления новой связи составляла 50% (`conn_add_prob = 0.5`), тогда как вероятность внедрения нового нейрона — 20% (`node_add_prob = 0.2`). Такая конфигурация позволяла алгоритму максимально использовать потенциал текущей размерности графа сети, прежде чем увеличивать ее вычислительную сложность.

4.5.4.3. Параметры популяции и критерии сходимости: Размер эволюционирующей популяции был задан на уровне 50 особей в каждом поколении (`pop_size = 50`). Оценка приспособленности базировалась на суммарной награде, получаемой агентом за удержание маятника без падения и выхода за границы допустимой зоны.

В качестве критерия раннего останова и успешного завершения обучения был установлен порог приспособленности `fitness_threshold = 500`, что соответствует максимально возможной оценке (продолжительности эпизода) в стандартной спецификации среды `CartPole-v1`. Максимальный горизонт эволюции был ограничен 50 поколениями, однако, как показали дальнейшие эксперименты, алгоритм успешно находил оптимальную топологию задолго до исчерпания этого лимита.

4.5.5. Архитектура и гиперпараметры агента DQN для среды CartPole

Программная реализация алгоритма Deep Q-Network (DQN) выполнялась с использованием библиотеки глубокого обучения PyTorch.

4.5.5.1. Архитектура нейронной сети: Для аппроксимации функции ценности действий (Q-функции) была спроектирована компактная полно-связная нейронная сеть прямого распространения (Multilayer Perceptron). Входной слой принимает вектор размерности 4, описывающий текущее состояние системы (координаты и скорости каретки и шеста).

Скрытый слой состоит всего из 16 нейронов с нелинейной функцией активации ReLU. Выбор столь малой размерности скрытого слоя обусловлен низкой сложностью пространства состояний среды CartPole-v1: компактная сеть позволяет избежать переобучения (overfitting) и существенно ускоряет процесс прямого и обратного распространения ошибки. Выходной слой представлен 2 нейронами с линейной активацией, каждый из которых генерирует предсказанное Q-значение для соответствующего дискретного действия (движение влево или вправо). Итоговое действие выбирается как аргумент максимального значения выходного вектора.

4.5.5.2. Стратегия исследования и дисконтирование: Для соблюдения баланса между исследованием среды и эксплуатацией уже известных выигрышных стратегий применялась ϵ -жадная политика.

- **Начальный уровень исследования** был задан как максимальный ($\epsilon = 1.0$), что заставляло агента на первых этапах действовать абсолютно случайно, накапливая разнообразный опыт.
- **Коэффициент затухания** ($\epsilon_{decay} = 0.995$) применялся на каждом шаге обучения, плавно снижая долю случайных действий до минимального порога $\epsilon_{min} = 0.01$.

- **Фактор дисконтирования** ($\gamma = 0.99$) обеспечивал высокую ”дальновидность” агента, заставляя его придавать большое значение будущим потенциальным наградам, что критически важно для задачи долгосрочной балансировки.

4.5.5.3. Буфер памяти и процесс оптимизации: Для стабилизации процесса обучения нейронной сети использовался механизм воспроизведения опыта (Experience Replay). Взаимодействия агента со средой сохранялись в циклический буфер памяти емкостью 10 000 переходов. На каждом шаге обучения (после накопления достаточного объема данных) из буфера случайным образом сэмплировался мини-батч размером 64 перехода. Это позволило разорвать временную корреляцию между последовательными состояниями и свести к минимуму дисперсию градиентов при обновлении весов.

Обновление весовых коэффициентов сети осуществлялось с помощью оптимизатора Adam со скоростью обучения 0.001. В качестве целевой метрики использовалась среднеквадратичная ошибка (MSE). Минимизируемая функция потерь L строилась на основе отклонения текущего предсказания сети от целевого значения, вычисляемого согласно уравнению Беллмана:

$$L = \frac{1}{N} \sum_{t=0}^T ((r_t + \gamma \max_{a'} Q(s_t, a')) - Q(s_t, a_t))^2 \quad (4.2)$$

где N – размер батча, s_t и a_t – текущие состояние и действие, r_t – полученная награда, s'_t – следующее состояние, а γ – коэффициент дисконтирования.

Критерием успешного решения среды служило достижение суммарной

награды в 500 баллов за один эпизод, что означает непрерывное удержание шеста в течение 500 тактов симуляции. По достижении этого порога веса модели (`state_dict`) автоматически сохранялись для последующей демонстрации и анализа.

4.6. Среда LunarLander (Посадка лунного модуля)

Среда LunarLander впервые упоминается в пакете сред Gym [4], которая представляет собой задачу управления в условиях двумерной физической симуляции с непрерывным пространством состояний. Задача когнитивного агента заключается в осуществлении безопасной и мягкой посадки космического модуля на заданную посадочную площадку, расположенную между двумя маркерными флажками. В отличие от задачи балансировки маятника, здесь агент должен научиться преодолевать гравитацию и управлять инерцией, аккуратно дозируя тягу главного и двух маневровых двигателей.

В рамках данного исследования для оценки способности алгоритмов к обобщению (устойчивости) тестирование проводилось в двух различных конфигурациях среды: в условиях идеального вакуума и с применением сильного бокового ветра и турбулентности. Сравнительный анализ влияния этих возмущающих факторов на стабильность агентов подробно рассматривается в разделе экспериментальных результатов.

4.6.1. Пространство состояний

Сенсорный аппарат агента передает вектор наблюдений, состоящий из 8 числовых параметров, исчерпывающе описывающих текущее кинемати-

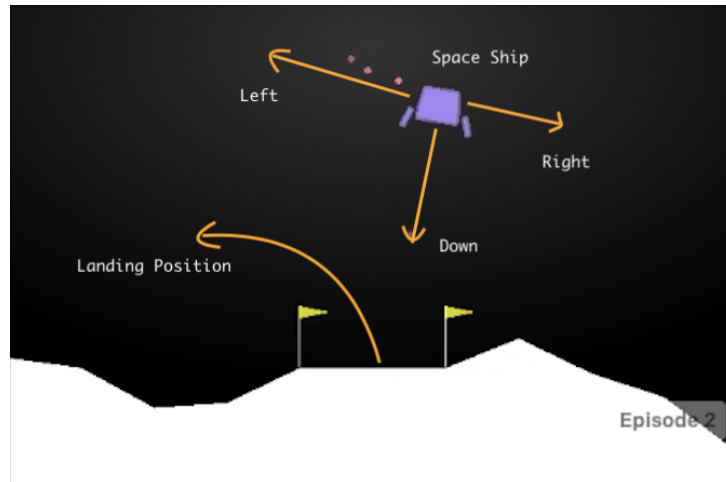


Рис. 4.3: Вид среды LunarLander.

ческое состояние модуля:

- **Позиция центра масс** аппарата в двумерной системе координат (x, y) .
- **Вектор линейной скорости** перемещения модуля по осям x и y .
- **Угол наклона (крена)** аппарата относительно идеальной вертикали.
- **Угловая скорость** вращения модуля.
- **Два дискретных булевых флага**, сигнализирующих о наличии физического контакта левой и правой посадочных опор с поверхностью.

4.6.2. Пространство действий

Пространство возможных решений агента является дискретным и состоит из 4 взаимоисключающих команд, выбираемых нейросетью на каждом такте симуляции:

- **0:** Не предпринимать никаких действий (свободное падение).
- **1:** Включить левый маневровый двигатель (для корректировки крена вправо).

- **2:** Включить главный (нижний) маршевый двигатель.
- **3:** Включить правый маневровый двигатель (для корректировки крена влево).

4.6.3. Функция награды и условия завершения

В отличие от бинарного сигнала выживания в CartPole, функция приспособленности в LunarLander имеет сложную композитную структуру, стимулирующую достижение цели при максимальной экономии ресурсов:

- За успешное касание поверхности обеими опорами агенту начисляется бонус в +100 очков.
- За движение к центру целевой площадки и корректное снижение скорости присуждается от +100 до +140 очков.
- В случае крушения модуля (падения на корпус) накладывается строгий штраф в размере -100 очков.
- Работа двигателей пенализируется для оптимизации расхода топлива: каждый такт работы главного двигателя стоит -0.3 очка, а использование боковых маневровых двигателей штрафует на -0.03 очка за такт.

Один эпизод симуляции принудительно обрывается при достижении лимита в 1000 шагов, а также при успешной посадке или крушении модуля. Идеальная, мягкая посадка в центр площадки позволяет набрать около 250–300 очков. Официальным порогом успешного решения среды, свидетельствующим о сходимости алгоритма, считается достижение средней награды в 200 баллов.

4.6.4. Конфигурация алгоритма NEAT для LunarLander

Файл конфигурации NEAT³ определяет фундаментальные правила эволюции. Ниже приведены ключевые параметры, использованные в экспериментах:

4.6.4.1. Основные параметры популяции и завершения:

- **Размер популяции (pop_size = 150):** Данный параметр устанавливает количество нейронных сетей (агентов) в каждом поколении. Значение 150 выбрано для обеспечения достаточного разнообразия геномов и эффективного поиска в пространстве решений.
- **Критерий приспособленности (fitness_criterion = max):** Эволюция стремится максимизировать функцию приспособленности, выбирая лучшие геномы для размножения.
- **Порог приспособленности (fitness_threshold = 200.0):** Обучение считается завершенным, если хотя бы один агент набирает 200.0 очков (стандартный порог решения среды).

4.6.4.2. Архитектура и инициализация генома:

- **Входы и Выходы:** Количество входных нейронов (num_inputs = 8) и выходных нейронов (num_outputs = 4) жестко зафиксировано и соответствует спецификации среды LunarLander.
- **Начальные скрытые слои (num_hidden = 0):** Поиск начинается с самых простых сетей, без скрытых нейронов, чтобы позволить алгоритму NEAT самостоятельно развить сложную топологию.

³<https://github.com/Isn0v/neat-training/blob/master/environments/lunar-lander/neat/neat.cfg>

- **Функции активации (`activation_default = tanh`):** Все нейроны используют функцию активации гиперболического тангенса ($\tanh$), которая позволяет вводить нелинейность в работу сети и выдает значения в диапазоне $[-1, 1]$, что подходит для управления двигателями.
- **Начальные связи (`initial_connection = full_direct`):** На старте каждый входной нейрон соединен со всеми выходными нейронами, что обеспечивает минимальную функциональность сети.

4.6.4.3. Вероятности мутации:

- **Мутация весов (`weight_mutate_rate = 0.8`):** С вероятностью 80% вес существующей связи будет изменен. Высокое значение стимулирует постоянное уточнение "тонкой настройки" управления.
- **Добавление нейрона/связи (`node_add_prob = 0.2`, `conn_add_prob = 0.5`):** Задают вероятности добавления нового скрытого нейрона (20%) или новой синаптической связи (50%). Это обеспечивает рост сложности топологии сети.
- **Удаление нейрона/связи (`node_delete_prob = 0.2`, `conn_delete_prob = 0.5`):** Задают вероятности удаления нейронов или связей, что позволяет упрощать избыточные структуры.

4.6.4.4. Специация и эволюция популяции:

- **Видообразование (`compatibility_threshold = 3.0`):** Порог совместимости δ в 3.0 очка определяет, насколько генетически различными должны быть сети, чтобы считаться разными видами.

- **Застой видов (`max_stagnation = 20`):** Если лучшая приспособленность вида не улучшается в течение 20 поколений, он считается застойным и не получает ресурсов для размножения. Это предотвращает ”зацикливание” эволюции на локальных максимумах.
- **Элитизм (`species_elitism = 2`):** Два лучших вида популяции защищены от удаления даже при застое.
- **Воспроизводство (`elitism = 2, survival_threshold = 0.2`):** В каждом виде два лучших генома гарантированно выживают, и только 20% наиболее приспособленных особей вида получают право на размножение.

4.6.5. Архитектура и гиперпараметры агента DQN для среды LunarLander

Программная реализация алгоритма DQN для задачи посадки лунного модуля базировалась на фреймворке `stable_baselines3`, рассмотренном в работе [7]. Он предоставляет оптимизированные и надежные реализации современных алгоритмов обучения с подкреплением. В связи с необходимостью комплексной оценки устойчивости когнитивных агентов, обучение проводилось в два этапа: формирование базовой политики управления⁴ и универсальной политики⁵ в условиях ветренности.

Процесс Q-обучения регулировался следующим набором гиперпараметров:

- **Буфер воспроизведения опыта (Replay Buffer):** Емкость буфера

⁴<https://github.com/Isn0v/neat-training/blob/master/environments/lunar-lander/rl/model.py>

⁵https://github.com/Isn0v/neat-training/blob/master/environments/lunar-lander/rl/model_windy.py

составила 100 000 переходов, что обеспечило эффективную декорреляцию обучающей выборки. Процесс обучения (сэмплирования из буфера) инициировался после накопления первых 1 000 случайных шагов (`learning_starts = 1000`).

- **Оптимизация и градиентный спуск:** Использовался размер пакета данных (`batch size`) равный 128. Скорость обучения (`learning rate`) была установлена на уровне 10^{-3} .
- **Параметры обновления:** Дисконтирующий множитель γ составил 0.99, ориентируя агента на долгосрочное планирование. Синхронизация весов целевой сети производилась каждые 250 шагов оптимизации.
- **Стратегия исследования среды:** Применялась ϵ -жадная стратегия с линейным затуханием. Начальное значение параметра исследования $\epsilon = 1.0$ постепенно снижалось до 0.05 на протяжении первых 10% от общего времени обучения (`exploration_fraction = 0.1`), после чего агент переходил к преимущественной эксплуатации полученных знаний.

4.7. Среда Ant (Ходьба муравья)

В качестве полигона для исследования возможностей нейроэволюции была выбрана среда непрерывного управления Ant-v4 из библиотеки Gymnasium. Данная среда представляет собой физическую симуляцию на базе движка MuJoCo, описанную в [8], для четырехногого агента («муравья»), цель которого – максимально быстро и эффективно продвигаться вперед (по оси X), сохраняя при этом баланс и минимизируя затраты

энергии на лишние движения.



Рис. 4.4: Схематическое изображение агента в среде Ant-v4.

4.7.1. Пространство состояний и действий

Сложность среды обусловлена физическими параметрами модели:

- **Пространство состояний (27 параметров):** включает в себя данные о положении корпуса в пространстве, углы поворота всех восьми сочленений, а также векторы их линейных и угловых скоростей. Такой объем данных необходим нейросети для понимания текущей позы робота и инерции его частей.
- **Пространство действий (8 параметров):** агент управляет крутящими моментами, прикладываемыми к каждому из восьми суставов. Действия являются непрерывными и лежат в диапазоне $[-1, 1]$.

4.7.2. Функция награды и условия завершения

Функция приспособленности (fitness) в данном эксперименте базируется на стандартной системе вознаграждений Ant-v4, которая включает:

1. **Reward for moving forward:** поощрение за продвижение вдоль оси X.
2. **Healthy reward:** фиксированный бонус за каждый шаг, пока робот остается «здоров» (не перевернулся и не вышел за пределы допустимых углов).
3. **Control cost:** штраф за слишком резкие или энергозатратные движения, что стимулирует агента к плавной и естественной походке.

Для оптимизации процесса эволюции была внедрена эвристика «жесткого таймера». Если в течение 50 шагов симуляции агент не продвигается вперед более чем на 0.05 метра, эпизод прерывается досрочно. Это позволяет алгоритму быстрее переходить к оценке следующей особи, не тратя время на агентов, которые застряли или совершают хаотичные движения на месте. Окончательная оценка генома вычисляется как среднее арифметическое результатов трех независимых эпизодов.

4.7.3. Конфигурация алгоритма NEAT для среды Ant-v4

Конфигурация NEAT⁶ для среды Ant-v4 потребовала тонкой настройки параметров эволюции, так как высокая размерность пространства действий (8 суставов) создает риск хаотичного изменения топологии, препятствующего обучению локомоции. Основные аспекты реализации разделены

⁶<https://github.com/Isn0v/neat-training/blob/master/environments/ant/neat.cfg>

на архитектурные параметры, генетические механизмы и инфраструктуру обучения.

4.7.3.1. Архитектурные параметры и инициализация: Начальная структура нейросетей была выбрана максимально упрощенной для минимизации вычислительной сложности на первых этапах:

- **Входной и выходной слои:** конфигурация сети жестко привязана к физике среды: 27 входных узлов (состояние) и 8 выходных узлов (моменты в суставах).
- **Начальная связность:** использован параметр `initial_connection = full_direct`, создающий связи между каждым входом и выходом без скрытых нейронов (`num_hidden = 0`). Это позволяет эволюции сначала оптимизировать линейные зависимости управления, прежде чем переходить к сложным нелинейным структурам.
- **Функции активации и агрегации:** для выходных сигналов выбран гиперболический тангенс (`tanh`), что идеально соответствует диапазону управления в MuJoCo $[-1, 1]$. Агрегация сигналов в узлах установлена как усреднение (`mean`), что обеспечивает более стабильный градиент активации при добавлении новых связей, в отличие от стандартного суммирования.

4.7.3.2. Генетические операторы и мутации:

- **Генетические операторы:** Для обеспечения баланса между эксплуатацией текущих решений и исследованием новых топологий были установлены следующие коэффициенты:

- **Мутация весов и смещений:** используется гауссово распределение со средним значением 0.0 и стандартным отклонением 0.1. Максимальные и минимальные значения весов ограничены диапазоном $[-3.0, 3.0]$ для предотвращения «взрыва» активаций в глубоких слоях эволюционирующей сети.
- **Структурные мутации:** вероятность добавления новой связи (`conn_add_prob`) составляет 0.3, а добавления нового узла (`node_add_prob`) – 0.1. При этом вероятности удаления компонентов (`conn_delete_prob`, `node_delete_prob`) установлены в 0, что гарантирует сохранение накопленной сложности топологии в рамках данного эксперимента. Данное решение было принято для ускорения сходимости, так как большинство полезных мутаций в Ant-v4 связано с добавлением новых связей, а не удалением существующих.

4.7.3.3. Видообразование и репродукция: Для защиты инноваций в популяции используется механизм разделения на виды:

- **Совместимость:** порог совместимости установлен на уровне 1.2, что позволяет разделять популяцию из 500 особей на устойчивые группы со схожими топологиями.
- **Элитизм и выживание:** в каждом виде сохраняется 1 лучшая особь (`species_elitism = 1`), а для всей популяции этот параметр равен 5. Порог выживания в 20% (`survival_threshold = 0.2`) гарантирует, что только наиболее приспособленные геномы будут участвовать в кроссовере для создания следующего поколения.

4.7.3.4. Инфраструктура обучения и отказоустойчивость: Техническая реализация в файле `model.py`⁷ включает ряд решений для обеспечения стабильности длительных вычислений:

- **Параллелизация:** расчет приспособленности популяции распределяется через `ParallelEvaluator` на все доступные ядра CPU, что сокращает время одного поколения в 4-8 раз.
- **Чекпоинты:** реализована система автоматического сохранения прогресса каждые 25 поколений. Это критически важно для среды Ant-v4, так как процесс обучения может занимать несколько часов и иногда даже дней.
- **Гибкое управление:** функция `run_neat` поддерживает восстановление из любого файла чекпоинта, что позволяет изменять параметры конфигурации «на лету» без потери накопленного прогресса эволюции.

5. Результаты исследования

В рамках данной выпускной квалификационной работы было проведено комплексное исследование применимости нейроэволюционных алгоритмов для задачи моделирования поведения когнитивных агентов. Главный вектор исследования был направлен на концептуальный и эмпирический сравнительный анализ эволюционного подхода (на базе алгоритма NEAT) с классическими методами глубокого обучения с подкреплением (на примере алгоритма DQN).

⁷<https://github.com/Isn0v/neat-training/blob/master/environments/ant/model.py>

Для максимально объективной оценки потенциала исследуемых алгоритмов была разработана методология поэтапного усложнения экспериментальных условий. Тестирование когнитивных моделей проводилось в трех симуляционных средах, каждая из которых предъявляла к агентам уникальные требования и обладала различной физикой взаимодействия:

1. **CartPole** — базовая задача поддержания динамического равновесия, выступившая в роли фундаментального полигона (baseline) для первичной проверки алгоритмов в дискретном пространстве действий.
2. **LunarLander** — комплексная задача пространственного маневрирования, позволившая оценить способности агентов к микроконтролю, а также протестировать их устойчивость в условиях добавления внешних возмущений (имитация ветровых потоков и турбулентности).
3. **Ant** — высокоуровневая задача непрерывного управления многокомпонентной биомеханической системой, продемонстрировавшая пределы применимости алгоритмов в многомерных пространствах.

Оценка эффективности обученных агентов производилась не только по классическому критерию максимизации получаемой награды, но и с учетом таких критически важных для реальных систем параметров, как архитектурная сложность нейронной сети, скорость схождения и устойчивость к шумовым воздействиям.

Полученные в ходе симуляций экспериментальные данные позволили сформулировать ряд существенных выводов, описывающих сильные и слабые стороны каждого из подходов. Ниже представлены детализированные результаты исследования, систематизированные по каждой из рассмотрен-

ных сред моделирования.

5.1. Результаты в среде CartPole

Первым этапом экспериментального исследования стала классическая задача управления — балансировка перевернутого маятника (CartPole). Данная среда выступала в качестве фундаментального полигона для верификации работоспособности алгоритмов и оценки базовых характеристик когнитивных агентов. В рамках эксперимента проводилось сопоставление классического алгоритма глубокого Q-обучения (DQN) и нейроэволюционного метода (NEAT).

На этапе проектирования архитектур моделей были применены два принципиально разных подхода. Агент на базе DQN использовал фиксированную полносвязную нейронную сеть с одним скрытым слоем, содержащую 114 параметров. В противовес этому, алгоритм NEAT функционировал в парадигме постепенного усложнения (complexifying), начиная эволюционный процесс с простейшей топологии без скрытых слоев и динамически формируя структуру сети в ответ на требования среды.

Сравнительный анализ метрик (согласно определениям 2.1-2.4) позволил сделать следующие ключевые выводы:

5.1.1. Эффективность использования выборки и скорость схождения

Оба алгоритма успешно справились с задачей, однако нейроэволюционный подход продемонстрировал существенно более высокую скорость адаптации. Алгоритму NEAT потребовалось всего 4 поколения (или 10 548

шагов взаимодействия со средой), чтобы найти оптимальную стратегию. В то же время алгоритму DQN для стабилизации весов и схождения потребовалось 471 эпизод, что эквивалентно 25 923 взаимодействиям. Таким образом, эволюционный алгоритм оказался более чем в 2,4 раза эффективнее с точки зрения затрат вычислительных тактов симуляции.

5.1.2. Качество решения и архитектурная сложность

В идеальных условиях обе модели достигли абсолютного максимума качества (средняя награда 500.00 из 500) со 100% стабильностью (стандартное отклонение равно 0). Однако способы достижения этого результата кардинально различаются. В то время как DQN использует плотную полносвязную структуру (плотность связей 100%), NEAT синтезировал высокоэффективную разреженную сеть, состоящую всего из 9 параметров (плотность связей 75%). Это доказывает способность нейроэволюции находить экстремально компактные и легковесные решения, что критически важно для развертывания моделей на устройствах с ограниченными вычислительными ресурсами.

5.1.3. Устойчивость к внешним возмущениям

Наиболее значимое различие между алгоритмами было зафиксировано при стресс-тестировании агентов в условиях зашумленных данных. Для генерации шума в этом и последующий проектах используется Гауссовский шум. В данной среде берется стандартное отклонение равное 0.1. При добавлении шума в наблюдения среды, эффективность полносвязной модели DQN резко снизилась на 29,14% (средняя награда упала до 354.30). Агент

оказался склонен к переобучению под «идеализированную» картину мира. Напротив, агент NEAT продемонстрировал абсолютную робастность: в условиях аналогичного зашумления модель не потеряла ни одного балла, сохранив среднюю награду на уровне 500.00. Столь высокая устойчивость напрямую вытекает из компактности эволюционной архитектуры: из-за отсутствия «лишних» связей и параметров, сеть физически лишена способности пропускать через себя шумовые сигналы, фильтруя их на уровне самой топологии.

5.1.4. Практические выводы

Эксперимент в среде CartPole эмпирически доказал, что для решения базовых задач когнитивного управления нейроэволюционный алгоритм NEAT превосходит классический DQN по скорости схождения, компактности получаемой модели и, что наиболее важно, по уровню устойчивости к неопределенностям среды.

5.2. Результаты в среде LunarLander

Эксперименты в среде LunarLander стали ключевым этапом практического исследования, позволившим оценить работу когнитивных агентов в условиях сложного пространства состояний и необходимости тонкого микроконтроля. В отличие от базовой задачи CartPole, здесь перед агентами стояла цель не просто удержания баланса, а поиска оптимальной траектории посадки в условиях меняющейся гравитации и внешних возмущений.

5.2.1. Сравнительный анализ архитектурной сложности и обучения

В ходе исследования был зафиксирован значительный контраст в подходах к формированию управляющей политики:

5.2.1.1. Информационная емкость: Алгоритм DQN для достижения стабильных результатов потребовал создания глубокой нейронной сети с архитектурой [512, 512], что составило 69 124 обучаемых параметра. В то же время алгоритм NEAT, благодаря механизму эволюции топологий, синтезировал компактные решения, содержащие в среднем от 150 до 200 параметров (связей и узлов). Плотность связей в моделях NEAT оказалась в сотни раз ниже, чем в полносвязных сетях DQN.

5.2.1.2. Скорость схождения: DQN продемонстрировал высокую эффективность обучения в «лабораторных» условиях, достигнув целевого порога приспособленности (>200) за 170 000 шагов взаимодействия со средой (примерно за 224 секунды астрономического времени). Нейроэволюционный подход потребовал значительно большего количества итераций симуляции, однако итоговые модели оказались на порядок экономичнее с точки зрения вычислительных ресурсов при инференсе.

5.2.2. Феномен «нейролоботомии» и проблема локальных оптимумов

В процессе обучения NEAT-агентов мы столкнулись со специфическим поведением, которое в рамках данной работы было классифицировано как «нейролоботомия».

Суть явления заключается в том, что в условиях жестких штрафов за

крушение (около -100 очков) эволюция на ранних этапах находит «безопасную», но деструктивную стратегию — полное отключение двигателей или минимизацию любых действий. Агент фактически «выключает» части своей нейронной сети (обрывает связи), чтобы избежать риска падения, предпочитая медленное свободное падение активному маневрированию. Это классический пример застревания в локальном оптимуме, где «неделание ничего» оказывается статистически выгоднее, чем «попытка управления», приводящая к аварии. Преодоление этого эффекта потребовало тонкой настройки функций приспособленности и введения механизмов рандомизации.

5.2.3. Устойчивость в условиях неопределенности и внешних шумов

Особое внимание было уделено устойчивости агентов к внешним шумам и погодным условиям (боковой ветер, турбулентность). Для этого были реализованы следующие программные решения:

- **Для DQN:** Разработана обертка UniversalWindWrapper, которая с вероятностью 50% инициировала эпизоды с экстремальным ветром (мощностью до 20.0). Это заставило сеть обучаться обобщенным паттернам компенсации дрейфа.
- **Для NEAT:** Внедрена система мульти-эпизодного тестирования (EPISODES_PER_GENOME = 5). Каждый геном проходил серию испытаний, где условия (штиль/ветер) чередовались. Итоговая оценка выставлялась как сумма достижений во всех 5 эпизодах, что исключало фактор случайного «везения» и отбирало только по-настоящему устойчивые топологии.

Результаты тестов на устойчивость показали, что универсальная модель DQN, хотя и сохраняет высокую среднюю награду в идеальных условиях (85.75), при появлении шумов в сенсорах снижает эффективность до 70.13. Модели NEAT, напротив, благодаря своей разреженной структуре, проявили «иммунитет» к высокочастотным помехам: отсутствие лишних связей не позволило шуму распространяться по сети, что обеспечило более стабильную посадку в турбулентных условиях.

5.2.4. Специфика программной реализации

Для обеспечения научной достоверности и воспроизводимости экспериментов были применены следующие подходы:

- **Seed Management:** Использование строго фиксированных наборов случайных чисел (seeds) для каждого поколения. Это гарантировало, что все 150-300 агентов внутри одной популяции соревнуются в абсолютно идентичных условиях (одинаковый наклон модуля, одинаковые порывы ветра), что делает отбор «честным».
- **Parallelization:** Ввиду высокой вычислительной сложности физики LunarLander и необходимости 5-кратного тестирования каждого агента, был использован модуль multiprocessing. Распределение вычислений по ядрам процессора позволило сократить время обучения.

5.2.5. Практические выводы

Таким образом, исследование в среде LunarLander подтвердило, что нейроэволюция (NEAT) является более предпочтительным методом для

создания устойчивых систем управления в условиях неопределенности и ограниченных вычислительных ресурсов (например, для встраиваемых систем посадочных модулей), в то время как DQN эффективнее для быстрого обучения в детерминированных средах с высокой размерностью данных.

5.3. Результаты в среде Ant

Экспериментальное исследование в среде симуляции четырехногого робота (Ant-v4) стало завершающим и наиболее технически сложным этапом практической части работы. Данная среда, базирующаяся на физическом движении MuJoCo, отличается высокоразмерным непрерывным пространством состояний и действий, что предъявляет экстремальные требования к архитектуре когнитивного агента.

5.3.1. Обоснование выбора модальности обучения

В отличие от сред CartPole и LunarLander, где проводился сравнительный анализ двух алгоритмов, моделирование в среде Ant-v4 осуществлялось исключительно с использованием нейроэволюционного подхода (NEAT). Данное ограничение было введено намеренно и обусловлено фундаментальными математическими ограничениями классического алгоритма глубокого Q-обучения (DQN).

Поскольку управление «муравьем» требует подачи непрерывных сигналов на 8 независимых суставов, применение DQN потребовало бы искусственной дискретизации пространства действий. Даже при минимальном разбиении (например, на 3 состояния: тянуть назад, покой, тянуть

вперед) алгоритм столкнулся бы с проблемой комбинаторного взрыва — $3^8 = 6561$ уникальная комбинация действий на каждом такте. Нейроэволюционный алгоритм NEAT, напротив, способен напрямую генерировать векторы непрерывных управляющих сигналов, что делает его оптимальным выбором для задач робототехники в рамках данного исследования.

5.3.2. Эволюция функции вознаграждения (Reward Shaping) и адаптация среды

Процесс обучения агента в среде Ant-v4 потребовал глубокого инженерного вмешательства в механику симуляции. В ходе первичных запусков с базовыми настройками среды был зафиксирован феномен стагнации приспособленности (отрицательные значения от -30 до 0). Анализ логов показал, что агент попадал в локальный оптимум, который можно охарактеризовать как «отказ от действий».

Для достижения итогового результата — биомеханически корректного и эффективного движения — был применен метод последовательного усложнения функции вознаграждения (Reward Shaping):

5.3.2.1. Решение проблемы «штрафа за энергию» (ctrl_cost): В базовой версии среды любые резкие сигналы на суставы карались жестким штрафом, который превышал награду за выживание. Эволюция NEAT реагировала на это радикально — алгоритм намеренно «ломал» нейросеть (уменьшая её размерность), чтобы выдавать нулевые сигналы и не получать штрафы. Для преодоления этой проблемы весовые коэффициенты штрафов в коде инициализации среды были снижены, что позволило агенту начать

первичные исследования моторики без риска мгновенного завершения эпизода.

5.3.2.2. Внедрение «умного таймера» стабилизации: Муравей появлялся в воздухе и падал на поверхность. Базовый код убивал агента за отсутствие скорости (таймер лени) еще до того, как тот успевал сгруппироваться после падения. Была разработана система «умного таймера», предоставляющая агенту запас времени (150 тактов) на стабилизацию после приземления и поощряющая любое, даже самое незначительное, продвижение вперед (установка собственного рекорда дистанции).

5.3.2.3. Получение биомеханически правильной походки: После того как агент научился не погибать на старте и хаотично перебирать конечностями (простая функция награды), система вознаграждения была переориентирована на максимизацию проекции скорости вектора центра масс по оси X при минимизации энергозатрат.

5.3.3. Практические выводы

Благодаря поэтапной модификации функции вознаграждения, алгоритм NEAT успешно синтезировал топологию нейронной сети, способную управлять сложной многозвенной системой.

Эволюционный процесс привел к отказу от хаотичных, судорожных движений в пользу скоординированной походки. Агент научился использовать инерцию собственного тела, синхронизируя работу 8 суставов для поддержания баланса и минимизации контакта корпуса с землей. Итоговая

модель нейросети оказалась поразительно компактной (учитывая сложность задачи), продемонстрировав способность нейроэволюции находить элегантные решения в многомерных пространствах без избыточного наращивания вычислительных мощностей.

6. Направления дальнейших исследований

Опираясь на полученные в ходе симуляций экспериментальные данные и подтвержденную эффективность нейроэволюционных подходов, был намечен ряд стратегических векторов для дальнейшего развития данного проекта. В частности, планируется масштабирование разработанных методов как в сторону усложнения архитектуры агентов, так и в сторону практического применения в прикладных симуляторах.

6.1. Разработка гибридных мультимодальных моделей (CNN + NEAT)

Первым направлением развития является преодоление ограничений классического алгоритма NEAT при работе со средами, обладающими высокой размерностью входных данных. В ходе исследования было отмечено, что базовая нейроэволюция испытывает существенные вычислительные трудности при обработке сырых пикселей (например, при получении визуальной информации напрямую с камер агента). Прямая подача изображений на вход NEAT приводит к комбинаторному взрыву возможных топологий и критическому замедлению эволюции.

Для решения этой проблемы планируется разработка и тестирование гибридной мультимодальной архитектуры. Суть подхода заключается в разделении задач:

- **Сверточные нейронные сети (CNN)** будут использоваться в качестве модуля «зрения» для автоматического извлечения репрезентативных визуальных признаков и сжатия размерности (понижения размерности картинки до компактного вектора данных).
- **Алгоритм NEAT** будет использоваться в качестве модуля «принятия решений», принимающего на вход обработанные признаки от CNN и формирующего итоговую управляющую политику.
- **Апробация данной связки** планируется провести на стандартизированном наборе визуальных сред — классических играх Atari, что позволит сравнить гибридную модель с эталонными решениями глубокого обучения.

6.2. Интеграция алгоритмов в институтский симулятор популяций

Второе направление представляет собой глобальную практическую цель, послужившую фундаментальной мотивацией для проведения данного дипломного исследования. В настоящее время на базе института ведется разработка комплексного симулятора поведения популяций виртуальных муравьев.

Внедрение алгоритмов машинного обучения в подобные масштабные проекты напрямую, «вслепую», несет высокие риски нестабильности си-

стемы. В связи с этим было критически важно провести предварительный изолированный анализ: проверить сильные и слабые стороны алгоритма NEAT на стандартизированных кинематических моделях (таких как среда непрерывного управления Ant-v4 в физическом движке MuJoCo), настроить функции вознаграждения и изучить поведение агентов вне влияния побочных факторов.

Результаты текущей работы сформировали готовую, эмпирически доказанную методологию. Мы на практике подтвердили два ключевых преимущества нейроэволюции, критически важных для симуляции популяций:

- **Легковесность моделей:** Компактность сформированных топологий позволяет запускать сотни и тысячи независимых агентов одновременно без перегрузки вычислительных мощностей системы (в отличие от «тяжелых» сетей DQN).
- **Устойчивость к зашумлению:** Разреженная архитектура сетей делает агентов робастными к изменениям внешней среды и непредсказуемым физическим взаимодействиям.

В качестве финального этапа развития проекта, протестированный и оптимизированный пайплайн обучения NEAT будет интегрирован в архитектуру институтского симулятора. Это позволит перейти от управления одиночными когнитивными агентами к моделированию сложных эмерджентных форм поведения в рамках целых популяций виртуальных насекомых.

Список литературы

1. Sutton R.S., Barto A.G. Reinforcement learning: An introduction. 2-е изд. The MIT Press, 2018 .
2. Mnih V. и др. Human-level control through deep reinforcement learning // Nature. Nature Publishing Group, a division of Macmillan Publishers Limited. All Rights Reserved., 2015. Т. 518, № 7540. С. 529–533.
3. Stanley K.O., Miikkulainen R. Evolving neural networks through augmenting topologies // Evol Comput. 2002. Т. 10, № 2. С. 99–127.
4. Towers M. и др. Gymnasium: A standard interface for reinforcement learning environments. 2025.
5. McIntyre A. и др. neat-python.
6. Barto A.G., Sutton R.S., Anderson C.W. Neuronlike adaptive elements that can solve difficult learning control problems // IEEE Trans Syst Man Cybern. 1983. Т. SMC-13, № 5. С. 834–846.
7. Raffin A. и др. Stable-baselines3: reliable reinforcement learning implementations // J. Mach. Learn. Res. JMLR.org, 2021. Т. 22, № 1. 8 с.
8. Todorov E., Erez T., Tassa Y. Mujoco: A physics engine for model-based control // Intelligent robots and systems (IROS), 2012 IEEE/RSJ international conference on. 2012. С. 5026–5033.